

.P4

.Json

Z3

▷ Parser logic
1: state parse ethernet:
transition select(ethernet.etherType):
MYTUNNEL → parse myTunnel
IPV4 → parse ipv4
default → accept
state parse myTunnel:
transition select(myTunnel.proto id):
IPV4 → parse ipv4
default → accept
state parse ipv4:
transition → accept

▷ Ingress tables
table ipv4 lpm:
key = ipv4.dstAddr (LPM)
actions = {ipv4 forward, drop}

▷ Ingress apply block
21: if ipv4 & !myTunnel:
apply(ipv4 lpm)
23: else if myTunnel:
apply(myTunnel exact)

```
parse_ethernet: {  
  transitions: [  
    { value: mytunnel, next_state: parse_myTunnel },  
    { value: ipv4, next_state: parse_ipv4 },  
    { default, next_state: accept }  
  ]  
  transition_key: [ethernet.etherType]  
}
```

```
ipv4_lpm: {  
  key: [hdr.ipv4.dstAddr],  
  match_kind: "lpm",  
  actions: [ipv4_forward, drop]  
}
```

Symbolic Path Constraint

```
etherType = BitVec('ethernet.etherType', 16)  
path_to_myTunnel = (etherType == mytunnel)  
path_to_ipv4 = (etherType == ipv4)  
default_path = Not(Or( etherType == mytunnel, etherType == ipv4))
```

Symbolic Table Model

```
ipv4_dstAddr = BitVec('hdr.ipv4.dstAddr', 32)  
mask = BitVecVal(0xFFFFFFFF, 32)           # /24 prefix mask  
prefix = BitVecVal(0x0A000000, 32)          # 10.0.0.0  
port = BitVecVal(1, 9)                      # forward to port 1  
default_port = BitVecVal(511, 9)            # drop  
  
ipv4_lpm_rule = If(  
  (ipv4_dstAddr & mask) == prefix,  
  port,  
  default_port  
)
```